

```
1 #%%
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import warnings
7
8 warnings.filterwarnings("ignore")
9 #%%
10 from sklearn.model_selection import train_test_split
11 from sklearn.preprocessing import LabelEncoder,
12     StandardScaler
13 from sklearn.linear_model import Perceptron
14 from sklearn.metrics import accuracy_score,
15     classification_report, confusion_matrix
16 import tensorflow as tf
17 from tensorflow.keras.models import Sequential
18 from tensorflow.keras.layers import Dense
19 from tensorflow.keras.layers import Dropout
20 from tensorflow.keras.utils import to_categorical
21 #%%
22 from google.colab import drive
23 drive.mount("/content/drive")
24 df = pd.read_csv("/content/drive/MyDrive/ANN/Iris.csv")
25 #%% md
26 #%%
27 df.head()
28 #%%
29 df.info()
30 #%%
31 df.duplicated().sum()
32 #%%
33 df.isnull().sum()
34 #%%
35 sns.pairplot(df, hue = "Species")
36 #%%
```

```
37 X = df.drop(columns = ["Species", "Id"])
38 y = df["Species"]
39 #%%
40 X.head()
41 #%%
42 y.head()
43 #%%
44 encoder = LabelEncoder()
45 y_int = encoder.fit_transform(y)
46 #%%
47 X_train, X_test, y_train, y_test = train_test_split(X,
    y_int, test_size = 0.20, random_state = 42, stratify=
    y_int)
48 #%%
49 scaler = StandardScaler()
50 X_train_scaled = scaler.fit_transform(X_train)
51 X_test_scaled = scaler.fit_transform(X_test)
52 #%%
53 # Perceptron
54 per = Perceptron(max_iter = 1000, random_state = 42)
55 per.fit(X_train_scaled, y_train)
56 #%%
57 y_pred_per = per.predict(X_test_scaled)
58 print(y_pred_per)
59 #%%
60 accuracy_score(y_test, y_pred_per)
61 #%%
62 print(classification_report(y_test, y_pred_per))
63 #%%
64 # ANN
65 y_train_cat = to_categorical(y_train, num_classes = 3)
66 y_test_cat = to_categorical(y_test, num_classes = 3)
67 #%%
68 model = Sequential([
69     # input layer
70     Dense(16, input_dim = 4, activation = "relu"),
71     # hiddent layer
72     Dense(8, activation = "relu"),
```

```
73     # output
74     Dense(3, activation = "softmax")
75 ])
76 ###
77 model.compile(optimizer = "adam", loss = "
    categorical_crossentropy", metrics = ["accuracy"])
78 ###
79 history = model.fit(X_train_scaled, y_train_cat,
80                     epochs = 100,
81                     batch_size = 8,
82                     validation_split = 0.2,
83                     verbose = 1)
84 ###
85 loss, acc = model.evaluate(X_test_scaled, y_test_cat,
86                             verbose=1)
87 print(acc)
88 ###
89 plt.figure(figsize = (10, 4))
90 plt.plot(history.history["accuracy"], label = "
    train_acc")
91 plt.plot(history.history["val_accuracy"], label = "
    train_acc")
92 ###
93
```